

Kernel machines - PART I

September, 2008
MLSS'08

Laboratoire d'Informatique, de Traitement de l'Information et des Systèmes

Stéphane Canu
stephane.canu@litislab.eu



Roadmap

1 Kernels and the learning problem

- Three learning problems
- Learning from data: the problem
- Kernelizing the linear regression
- Kernel machines: a definition

2 Tools: the functional framework

- In the beginning was the kernel
- Kernel and hypothesis set
- Optimization, loss function and the regularization cost

3 Kernel machines

- Non sparse kernel machines
- Sparse kernel machines
- practical SVM

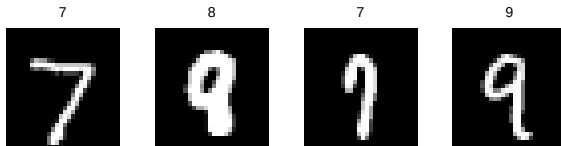
4 Conclusion



Optical character recognition

Example (The MNIST database)

- ▶ MNIST¹, data = « image-label »
- ▶ $n = 60,000$; $d = 700$; classes = 10
- ▶ Kernel error rate = 0.56 %,
- ▶ Best error rate = 0.4 % .



¹<http://yann.lecun.com/exdb/mnist/index.html>

```

<?xml version="1.0" encoding="iso-8859-1" ?>
<newsite itemid="2330" id="root" date="1996-08-20" xml:lang="en">
<title>USA: Tylan stock jumps; weighs in</title>
<headline>Tylan stock jumps; weighs in</headline>
<dateline>SAN DIEGO</dateline>
<text>
<p>The stock of Tylan General Inc. jumped after the
process-management equipment said it had signed a deal with
company and added that it has already signed a deal with
potential buyers.</p>
<p>Tylan was up $2.50 to $12.75 in early trading.
<p>The company said it has set up a committee to review
the sale and that Goldman, Sachs & Co. is its
financial adviser.</p>
</text>
<copyright>(c) Reuters Limited 1996</copyright>
<metadata>
<codes class="bip:countries:1.0">
<code code="USA"> </code>
</codes>
<codes class="bip:industries:1.0">
<code code="I34420"> </code>
</codes>
<codes class="bip:topics:1.0">
<code code="C15"> </code>
<code code="C152"> </code>
<code code="C18"> </code>
<code code="C181"> </code>
<code code="CCAT"> </code>
</codes>
<dc element="dc:publi." value="Reuters"/>
...
</metadata>

```

Example (Reuter RCV1)

- ▶ 800,000 texts (split into 20 categories (multiclass))
- ▶ Tasks: information retrieval
 - ▶ supervised
 - ▶ document classification
 - ▶ propose a classification
- ▶ results
 - ▶ linear kernels
 - ▶ deal with 1000000 / 1000000
 - ▶ Kernel rank reduction

^a<http://www.daviddlewis.com>

^bJoachims 2002

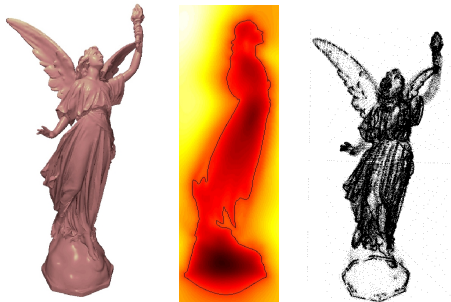
Example (Reuter RCV 1 database^a)

- ▶ 800,000 texts (short articles) and 1,000 categories (multilabelling).
- ▶ Tasks: information retrieval
 - ▶ supervised indexing
 - ▶ document clustering
 - ▶ propose a concept hierarchy
- ▶ results
 - ▶ linear kernels: the best^b
 - ▶ deal with 1,000 categories: one class / Kernels on the output
 - ▶ Kernel ranking

^a<http://www.daviddlewis.com/resources/testcollections/rcv1/>
^bJoachims 2002



Implicit Surface Modelling



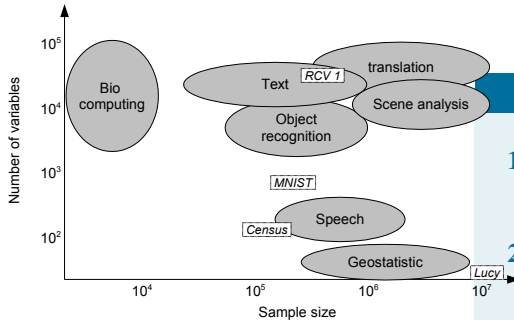
Example (*Lucy*²)

- ▶ 14 million points with normals.
- ▶ 364,982 compactly supported basis function centres.
- ▶ kernel regression with *thin plate splines* kernels

²<http://www.kyb.mpg.de/~walder>



Learning challenges: the size effect



3 key issues

1. learn any problem:
 - ▶ functional universality
2. from data:
 - ▶ statistical consistency
3. with large data sets:
 - ▶ computational efficiency

kernel machines address these three issues
(up to a certain point regarding efficiency)

Roadmap

1 Kernels and the learning problem

- Three learning problems
- Learning from data: the problem
- Kernelizing the linear regression
- Kernel machines: a definition

2 Tools: the functional framework

- In the beginning was the kernel
- Kernel and hypothesis set
- Optimization, loss function and the regularization cost

3 Kernel machines

- Non sparse kernel machines
- Sparse kernel machines
- practical SVM

4 Conclusion

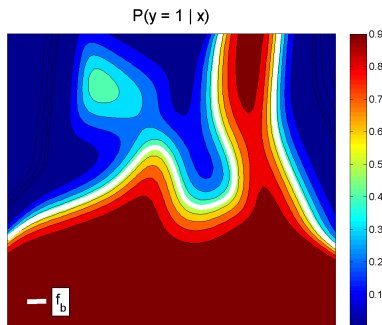
Machine learning: the problem

- ▶ input: $\mathbf{x} \in \mathcal{X}$ the domaine
- ▶ output: $y \in \mathcal{Y}$ the codomaine
- ▶ target: $f \in \mathcal{D}$ the set of measurable functions from $\mathcal{X}$ to $\mathcal{Y}$
- ▶ objective: loss $\ell: \mathcal{X} \times \mathcal{Y} \times \mathcal{D} \rightarrow \mathbb{R}$
 $(\mathbf{x}, y, f) \mapsto \ell(y, f(\mathbf{x}))$

	input Domaine $\mathcal{X}$	output Codomaine $\mathcal{Y}$	loss ℓ
MNIST	images 28×28 pixels	$\{0, 1, \dots, 9\}$	0 if $y = f(\mathbf{x})$ 1 else
RCV 1	XML files in NewsML text set	103 categories Reuters topic codes	0/1
Lucy	$\mathbb{R}^6$	$\mathbb{R}$	$(f(\mathbf{x}) - y)^2$

- ▶ problem: $\mathbb{P}(\mathbf{x}, y) \in \mathcal{P}$ a set of probability measures on $\mathcal{X} \times \mathcal{Y}$

The learning problem before data



let $\mathcal{X}, \mathcal{Y}, \mathcal{D}, \mathcal{P}$ be sets

Given:

► objective: loss

$$\begin{aligned} \ell: \mathcal{X} \times \mathcal{Y} \times \mathcal{D} &\rightarrow \mathbb{R} \\ (\mathbf{x}, y, f) &\mapsto \ell(y, f(\mathbf{x})) \end{aligned}$$

► problem: $\mathbb{P} \in \mathcal{P}$ s.t. $(X, Y) \sim \mathbb{P}$

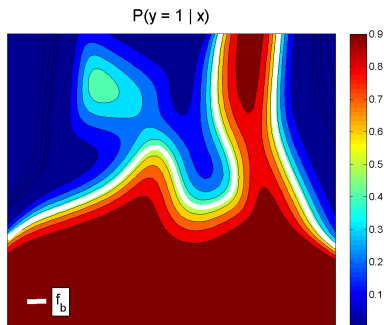
Find the function f_b

$$f_b = \operatorname{argmin}_{f \in \mathcal{D}} \underbrace{\mathbb{E}(\ell(Y, f(X)))}_{\text{Risk } R(f)}$$

$\mathbb{P}$ specify the problem $\implies$ function f_b is the target

$\mathbb{P}$ and f_b are unknown

The learning problem before data



let $\mathcal{X}, \mathcal{Y}, \mathcal{D}, \mathcal{P}$ be sets

Given:

► objective: loss

$$\begin{aligned} \ell: \mathcal{X} \times \mathcal{Y} \times \mathcal{D} &\rightarrow \mathbb{R} \\ (\mathbf{x}, y, f) &\mapsto \ell(y, f(\mathbf{x})) \end{aligned}$$

► problem: $\mathbb{P} \in \mathcal{P}$ s.t. $(X, Y) \sim \mathbb{P}$

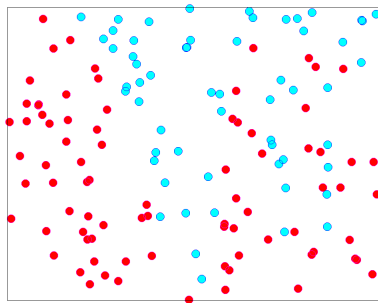
Find the function f_b

$$f_b = \operatorname{argmin}_{f \in \mathcal{D}} \underbrace{\mathbb{E}(\ell(Y, f(X)))}_{\text{Risk } R(f)}$$

$\mathbb{P}$ specify the problem $\implies$ function f_b is the target

$\mathbb{P}$ and f_b are unknown

Learning problem with data: functional estimation



- ▶ let $\ell(y, f(\mathbf{x}))$ be a loss
- ▶ $\mathbb{P}(\mathbf{x}, y) \in \mathcal{P}$ is **unknown**

Given:

- ▶ data $S_n = \{(\mathbf{x}_i, y_i), i = 1, n\}$ i.i.d. according to $\mathbb{P}$

Learning = Estimate function f_b

- ▶ Build an algorithm

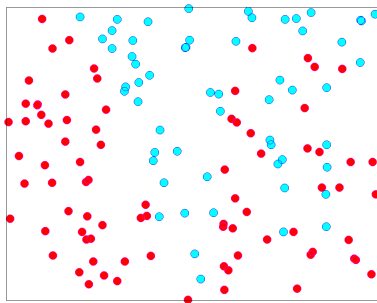
$$\begin{aligned} \mathcal{A}: (\mathcal{X}, \mathcal{Y})^n &\rightarrow \mathcal{H} \subset \mathcal{D} \\ S_n &\mapsto f = \mathcal{A}(S_n) \end{aligned}$$

Such that:

$$\begin{aligned} &\forall \mathbb{P} \in \mathcal{P} \\ &\forall S_n \text{ i.i.d.} \end{aligned} : \underbrace{\mathbb{E}(\ell(Y, \mathcal{A}))}_{\text{risk } f=\mathcal{A}(S_n)} \xrightarrow{n \rightarrow \infty} \underbrace{\mathbb{E}(\ell(Y, f_b(X)))}_{\text{lowest risk}} \quad (\text{in probability})$$

Requirements on $\mathcal{A}$: Universal consistency (+ efficiency)

Learning problem with data: functional estimation



- ▶ let $\ell(y, f(\mathbf{x}))$ be a loss
- ▶ $\mathbb{P}(\mathbf{x}, y) \in \mathcal{P}$ is **unknown**

Given:

- ▶ data $S_n = \{(\mathbf{x}_i, y_i), i = 1, n\}$ i.i.d. according to $\mathbb{P}$

Learning = Estimate function f_b

- ▶ Build an algorithm

$$\begin{aligned} \mathcal{A}: (\mathcal{X}, \mathcal{Y})^n &\rightarrow \mathcal{H} \subset \mathcal{D} \\ S_n &\mapsto f = \mathcal{A}(S_n) \end{aligned}$$

Such that:

$$\begin{aligned} &\forall \mathbb{P} \in \mathcal{P} \\ &\forall S_n \text{ i.i.d.} \end{aligned} : \underbrace{\mathbb{E}(\ell(Y, \mathcal{A}))}_{\text{risk } f=\mathcal{A}(S_n)} \xrightarrow{n \rightarrow \infty} \underbrace{\mathbb{E}(\ell(Y, f_b(X)))}_{\text{lowest risk}} \quad (\text{in probability})$$

Requirements on $\mathcal{A}$: Universal consistency (+ efficiency)



How to build an algorithm meeting these requirements?

intuition: a mean (may) converges towards its expectation

use data: the empirical risk (or a surrogate)

$$\begin{aligned}
 R(f) &= \mathbb{E}(\ell(Y, f(X))) \\
 &= \int \int \ell(y, f(x)) \mathbb{P}(x, y) dx dy
 \end{aligned}
 \qquad
 R_{\text{emp}}(f) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(\mathbf{x}_i))$$

theoretical	empirical (real stuff)
$\mathbb{P}$ unavailable	use R_{emp}
$\mathcal{D}$ unavailable	use a tractable hypothesis set $\mathcal{H}$
$f_b = \operatorname{argmin}_{f \in \mathcal{D}} R(f)$	$\mathcal{A}(S_n) = \operatorname{argmin}_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(\mathbf{x}_i))$

How to choose the set of hypothesis $\mathcal{H}$?



How to build an algorithm meeting these requirements?

intuition: a mean (may) converges towards its expectation

use data: the empirical risk (or a surrogate)

$$\begin{aligned}
 R(f) &= \mathbb{E}(\ell(Y, f(X))) \\
 &= \int \int \ell(y, f(x)) \mathbb{P}(x, y) dx dy
 \end{aligned}
 \qquad
 R_{\text{emp}}(f) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(\mathbf{x}_i))$$

theoretical	empirical (real stuff)
$\mathbb{P}$ unavailable	use R_{emp}
$\mathcal{D}$ unavailable	use a tractable hypothesis set $\mathcal{H}$
$f_b = \operatorname{argmin}_{f \in \mathcal{D}} R(f)$	$\mathcal{A}(S_n) = \operatorname{argmin}_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(\mathbf{x}_i))$

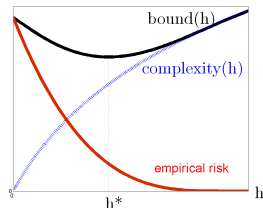
How to choose the set of hypothesis $\mathcal{H}$?

Machine Learning: universal functional estimation

problem about the hypothesis set

- ▶ if $\mathcal{H}$ is too small: no universality
- ▶ if $\mathcal{H}$ is too large: no consistency

solution: control $\mathcal{H}$ complexity



a general bound (with high probability)

$$\forall f \in \mathcal{H}$$

$$\forall \mathbb{P} \in \mathcal{P}$$

$$\forall S_n \sim \mathbb{P}$$

$$R(f) \leq R_{\text{emp}}(f) + \phi(n, h)$$

where h measures $\mathcal{H}$ complexity

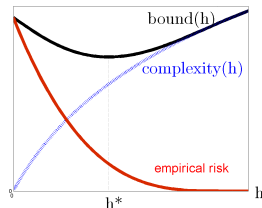
Learning principle: $\min_{f \in \mathcal{H}} \left(R_{\text{emp}} \text{ as well as } \mathcal{H} \text{ complexity} \right)$

Machine Learning: universal functional estimation

problem about the hypothesis set

- ▶ if $\mathcal{H}$ is too small: no universality
- ▶ if $\mathcal{H}$ is too large: no consistency

solution: control $\mathcal{H}$ complexity



a general bound (with high probability)

$$\forall f \in \mathcal{H}$$

$$\forall \mathbb{P} \in \mathcal{P}$$

$$\forall S_n \sim \mathbb{P}$$

$$R(f) \leq R_{\text{emp}}(f) + \phi(n, h)$$

where h measures $\mathcal{H}$ complexity

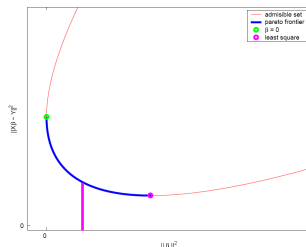
Learning principle: $\min_{f \in \mathcal{H}} \left(R_{\text{emp}} \text{ as well as } \mathcal{H} \text{ complexity} \right)$

3 equivalent formulations

$$b \left\{ \begin{array}{l} \min_{f \in \mathcal{H}} \sum_{i=1}^n \ell(f, \mathbf{x}_i, y_i) \\ \text{s.t. } R(f) \leq C \end{array} \right.$$

$$\left\{ \begin{array}{l} \min_{f \in \mathcal{H}} R(f) \\ \text{s.t. } \sum_{i=1}^n \ell(f, \mathbf{x}_i, y_i) \leq C' \end{array} \right.$$

$$\min_{f \in \mathcal{H}} \sum_{i=1}^n \ell(f, \mathbf{x}_i, y_i) + \lambda R(f)$$



BEWARE! convexity is required

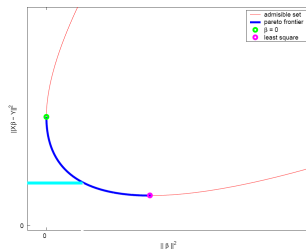
This is a multicriteria optimization task

3 equivalent formulations

$$b \left\{ \begin{array}{l} \min_{f \in \mathcal{H}} \sum_{i=1}^n \ell(f, \mathbf{x}_i, y_i) \\ \text{s.t. } R(f) \leq C \end{array} \right.$$

$$\left\{ \begin{array}{l} \min_{f \in \mathcal{H}} R(f) \\ \text{s.t. } \sum_{i=1}^n \ell(f, \mathbf{x}_i, y_i) \leq C' \end{array} \right.$$

$$\min_{f \in \mathcal{H}} \sum_{i=1}^n \ell(f, \mathbf{x}_i, y_i) + \lambda R(f)$$



BEWARE! convexity is required

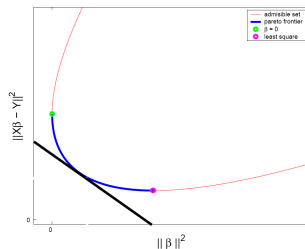
This is a multicriteria optimization task

3 equivalent formulations

$$b \left\{ \begin{array}{l} \min_{f \in \mathcal{H}} \sum_{i=1}^n \ell(f, \mathbf{x}_i, y_i) \\ \text{s.t. } R(f) \leq C \end{array} \right.$$

$$\left\{ \begin{array}{l} \min_{f \in \mathcal{H}} R(f) \\ \text{s.t. } \sum_{i=1}^n \ell(f, \mathbf{x}_i, y_i) \leq C' \end{array} \right.$$

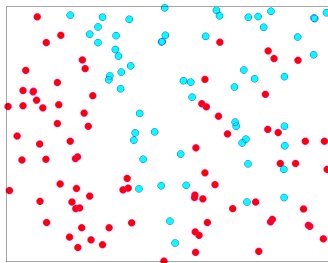
$$\min_{f \in \mathcal{H}} \sum_{i=1}^n \ell(f, \mathbf{x}_i, y_i) + \lambda R(f)$$



BEWARE! convexity is required

This is a multicriteria optimization task

Kernel machines intuition



- ▶ (1) fit the data
- ▶ (2) regular solution
- ▶ (3) examples contribution
- ▶ (4) remove useless examples

minimize R_{emp}

regularize

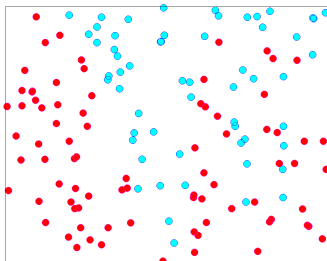
use **Kernels** $\iff$ specify $\mathcal{H}$

be sparse

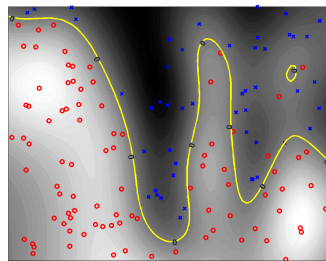
Kernel machines: $\min_{f \in \mathcal{H}} \left(R_{\text{emp}} \text{ and } \mathcal{H} \text{ complexity} \right)$ with kernel based $\mathcal{H}$

$\mathcal{A}(S_n)$ has to be universal and good statisticaly and computationaly

Kernel machines intuition



- ▶ (1) fit the data
- ▶ (2) regular solution
- ▶ (3) examples contribution
- ▶ (4) remove useless examples



minimize R_{emp}

regularize

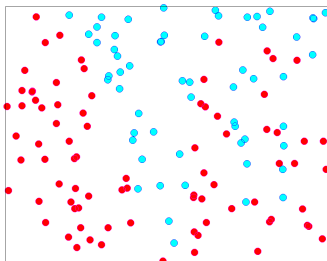
use **Kernels** $\iff$ specify $\mathcal{H}$

be sparse

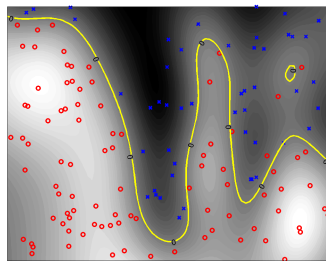
Kernel machines: $\min_{f \in \mathcal{H}} (R_{\text{emp}} \text{ and } \mathcal{H} \text{ complexity})$ with kernel based $\mathcal{H}$

$\mathcal{A}(S_n)$ has to be universal and good statisticaly and computationaly

Kernel machines intuition



- ▶ (1) fit the data
- ▶ (2) regular solution
- ▶ (3) examples contribution
- ▶ (4) remove useless examples



minimize R_{emp}

regularize

use **Kernels** $\iff$ specify $\mathcal{H}$

be sparse

Kernel machines: $\min_{f \in \mathcal{H}} \left(R_{\text{emp}} \text{ and } \mathcal{H} \text{ complexity} \right)$ with kernel based $\mathcal{H}$

$\mathcal{A}(S_n)$ has to be universal and good statisticaly and computationaly

Plan

1 Kernels and the learning problem

- Three learning problems
- Learning from data: the problem
- Kernelizing the linear regression
- Kernel machines: a definition

2 Tools: the functional framework

- In the beginning was the kernel
- Kernel and hypothesis set
- Optimization, loss function and the regularization cost

3 Kernel machines

- Non sparse kernel machines
- Sparse kernel machines
- practical SVM

4 Conclusion



the example of the linear least mean square

the linear model

$$y_i = \sum_{j=1}^d \beta_j x_{ij} + \varepsilon_i \quad , \quad i = 1, n$$

n observations and d variables; $d < n$

$$\min_{\beta} = \sum_{i=1}^n \left(\sum_{j=1}^d x_{ij} \beta_j - y_i \right)^2 = \|X\beta - Y\|^2$$

Solution: $\hat{\beta} = (X^T X)^{-1} X^T Y$

$$f(\mathbf{x}) = \mathbf{x}^T \underbrace{(X^T X)^{-1} X^T Y}_{\hat{\beta}}$$

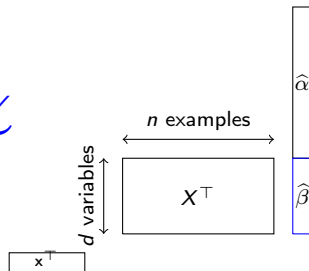
What is the influence of each example (X rows)?



The influence of the examples

for a new input $\mathbf{x}$

$$\begin{aligned}
 f(\mathbf{x}) &= \mathbf{x}^\top (X^\top X)(X^\top X)^{-1} \underbrace{(X^\top X)^{-1} X^\top Y}_{\hat{\beta}} \\
 &= \mathbf{x}^\top X^\top \underbrace{X(X^\top X)^{-1} (X^\top X)^{-1} X^\top Y}_{\hat{\alpha}}
 \end{aligned}$$



$$f(\mathbf{x}) = \sum_{j=1}^d \hat{\beta}_j x_j$$

from variables to examples

$$\underbrace{\hat{\alpha} = X(X^\top X)^{-1} \hat{\beta}}_{n \text{ examples}}$$

and

$$\underbrace{\hat{\beta} = X^\top \hat{\alpha}}_{d \text{ variables}}$$

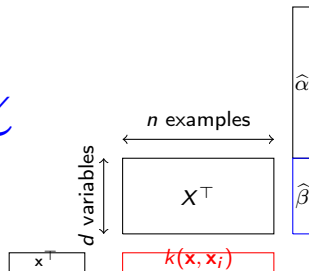
What if $d \geq n$?



The influence of the examples

for a new input $\mathbf{x}$

$$\begin{aligned}
 f(\mathbf{x}) &= \mathbf{x}^\top (X^\top X)(X^\top X)^{-1} \underbrace{(X^\top X)^{-1} X^\top Y}_{\hat{\beta}} \\
 &= \mathbf{x}^\top X^\top \underbrace{X(X^\top X)^{-1} (X^\top X)^{-1} X^\top Y}_{\hat{\alpha}}
 \end{aligned}$$



$$f(\mathbf{x}) = \sum_{j=1}^d \hat{\beta}_j x_j = \sum_{i=1}^n \hat{\alpha}_i (\mathbf{x}^\top \mathbf{x}_i)$$

from variables to examples

$$\underbrace{\hat{\alpha} = X(X^\top X)^{-1} \hat{\beta}}_{n \text{ examples}}$$

and

$$\underbrace{\hat{\beta} = X^\top \hat{\alpha}}_{d \text{ variables}}$$

What if $d \geq n$?

Non linear case: dictionnary vs. kernel

in the non linear case: use a **dictionary** of functions

$$\phi_j(\mathbf{x}), j = 1, p \quad \text{with possibly} \quad p = \infty$$

for instance polynomials, wavelets... (assume orthogonality)

$$f(\mathbf{x}) = \sum_{j=1}^p \hat{\beta}_j \phi_j(\mathbf{x}) \quad \text{with} \quad \hat{\beta}_j = \sum_{i=1}^n y_i \phi_j(\mathbf{x}_i)$$

using linearity

$$f(\mathbf{x}) = \sum_{i=1}^n y_i \underbrace{\sum_{j=1}^p \phi_j(\mathbf{x}_i) \phi_j(\mathbf{x})}_{k(\mathbf{x}_i, \mathbf{x})}$$

$$p \geq n \text{ so what since } k(\mathbf{x}_i, \mathbf{x}) = \sum_{j=1}^p \phi_j(\mathbf{x}_i) \phi_j(\mathbf{x})$$

Non linear case: dictionnairy vs. kernel

in the non linear case: use a **dictionary** of functions

$$\phi_j(\mathbf{x}), j = 1, p \quad \text{with possibly} \quad p = \infty$$

for instance polynomials, wavelets... (assume orthogonality)

$$f(\mathbf{x}) = \sum_{j=1}^p \hat{\beta}_j \phi_j(\mathbf{x}) \quad \text{with} \quad \hat{\beta}_j = \sum_{i=1}^n y_i \phi_j(\mathbf{x}_i)$$

using linearity

$$f(\mathbf{x}) = \sum_{i=1}^n y_i \underbrace{\sum_{j=1}^p \phi_j(\mathbf{x}_i) \phi_j(\mathbf{x})}_{k(\mathbf{x}_i, \mathbf{x})}$$

$$p \geq n \text{ so what since } k(\mathbf{x}_i, \mathbf{x}) = \sum_{j=1}^p \phi_j(\mathbf{x}_i) \phi_j(\mathbf{x})$$



closed form kernel: the quadratic kernel

The quadratic dictionary in $\mathbb{R}^d$:

$$\begin{aligned}\Phi: \mathbb{R}^d &\rightarrow \mathbb{R}^{p=1+d+\frac{d(d+1)}{2}} \\ \mathbf{s} &\mapsto \Phi = (1, s_1, s_2, \dots, s_d, s_1^2, s_2^2, \dots, s_d^2, \dots, s_i s_j, \dots)\end{aligned}$$

in this case

$$\Phi(\mathbf{s})^\top \Phi(\mathbf{t}) = 1 + s_1 t_1 + s_2 t_2 + \dots + s_d t_d + s_1^2 t_1^2 + \dots + s_d^2 t_d^2 + \dots + s_i s_j t_i t_j + \dots$$

The quadratic kernel: $\mathbf{s}, \mathbf{t} \in \mathbb{R}^d, \quad k(\mathbf{s}, \mathbf{t}) = (\mathbf{s}^\top \mathbf{t} + 1)^2$
 $= 1 + 2\mathbf{s}^\top \mathbf{t} + (\mathbf{s}^\top \mathbf{t})^2$

computes the dot product of the reweighted dictionary:

$$\begin{aligned}\Phi: \mathbb{R}^d &\rightarrow \mathbb{R}^{p=1+d+\frac{d(d+1)}{2}} \\ \mathbf{s} &\mapsto \Phi = (1, \sqrt{2}s_1, \sqrt{2}s_2, \dots, \sqrt{2}s_d, s_1^2, s_2^2, \dots, s_d^2, \dots, \sqrt{2}s_i s_j, \dots)\end{aligned}$$

$p = 1 + d + \frac{d(d+1)}{2}$ multiplications vs. $d + 1$
 use kernel to save computation



closed form kernel: the quadratic kernel

The quadratic dictionary in $\mathbb{R}^d$:

$$\begin{aligned}\Phi : \mathbb{R}^d &\rightarrow \mathbb{R}^{p=1+d+\frac{d(d+1)}{2}} \\ \mathbf{s} &\mapsto \Phi = (1, s_1, s_2, \dots, s_d, s_1^2, s_2^2, \dots, s_d^2, \dots, s_i s_j, \dots)\end{aligned}$$

in this case

$$\Phi(\mathbf{s})^\top \Phi(\mathbf{t}) = 1 + s_1 t_1 + s_2 t_2 + \dots + s_d t_d + s_1^2 t_1^2 + \dots + s_d^2 t_d^2 + \dots + s_i s_j t_i t_j + \dots$$

The quadratic kernel: $\mathbf{s}, \mathbf{t} \in \mathbb{R}^d, \quad k(\mathbf{s}, \mathbf{t}) = (\mathbf{s}^\top \mathbf{t} + 1)^2$
 $= 1 + 2\mathbf{s}^\top \mathbf{t} + (\mathbf{s}^\top \mathbf{t})^2$

computes the dot product of the reweighted dictionary:

$$\begin{aligned}\Phi : \mathbb{R}^d &\rightarrow \mathbb{R}^{p=1+d+\frac{d(d+1)}{2}} \\ \mathbf{s} &\mapsto \Phi = (1, \sqrt{2}s_1, \sqrt{2}s_2, \dots, \sqrt{2}s_d, s_1^2, s_2^2, \dots, s_d^2, \dots, \sqrt{2}s_i s_j, \dots)\end{aligned}$$

$p = 1 + d + \frac{d(d+1)}{2}$ multiplications vs. $d + 1$
 use kernel to save computation

closed form kernel: the quadratic kernel

The quadratic dictionary in $\mathbb{R}^d$:

$$\begin{aligned}\Phi : \mathbb{R}^d &\rightarrow \mathbb{R}^{p=1+d+\frac{d(d+1)}{2}} \\ \mathbf{s} &\mapsto \Phi = (1, s_1, s_2, \dots, s_d, s_1^2, s_2^2, \dots, s_d^2, \dots, s_i s_j, \dots)\end{aligned}$$

in this case

$$\Phi(\mathbf{s})^\top \Phi(\mathbf{t}) = 1 + s_1 t_1 + s_2 t_2 + \dots + s_d t_d + s_1^2 t_1^2 + \dots + s_d^2 t_d^2 + \dots + s_i s_j t_i t_j + \dots$$

The quadratic kernel: $\mathbf{s}, \mathbf{t} \in \mathbb{R}^d, \quad k(\mathbf{s}, \mathbf{t}) = (\mathbf{s}^\top \mathbf{t} + 1)^2$
 $= 1 + 2\mathbf{s}^\top \mathbf{t} + (\mathbf{s}^\top \mathbf{t})^2$

computes the dot product of the reweighted dictionary:

$$\begin{aligned}\Phi : \mathbb{R}^d &\rightarrow \mathbb{R}^{p=1+d+\frac{d(d+1)}{2}} \\ \mathbf{s} &\mapsto \Phi = (1, \sqrt{2}s_1, \sqrt{2}s_2, \dots, \sqrt{2}s_d, s_1^2, s_2^2, \dots, s_d^2, \dots, \sqrt{2}s_i s_j, \dots)\end{aligned}$$

$p = 1 + d + \frac{d(d+1)}{2}$ multiplications vs. $d + 1$
 use kernel to save computation



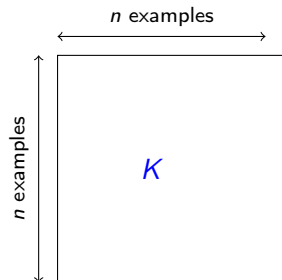
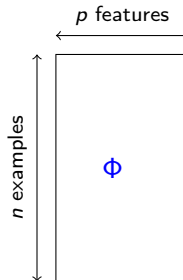
kernel: features through pairwise comparisons

$\mathbf{x}$

e.g. a text

$\phi(\mathbf{x})$

e.g. BOW



$$k(\mathbf{x}_i, \mathbf{x}_j) = \sum_{j=1}^p \phi_j(\mathbf{x}_i) \phi_j(\mathbf{x}_j)$$

K The matrix of pairwise comparisons ($\mathcal{O}(n^2)$)

Roadmap

1 Kernels and the learning problem

- Three learning problems
- Learning from data: the problem
- Kernelizing the linear regression
- Kernel machines: a definition

2 Tools: the functional framework

- In the beginning was the kernel
- Kernel and hypothesis set
- Optimization, loss function and the regularization cost

3 Kernel machines

- Non sparse kernel machines
- Sparse kernel machines
- practical SVM

4 Conclusion

Kernel machines

use a kernel as a dictionary

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i k(\mathbf{x}, \mathbf{x}_i)$$

► α_i influence of example i

depends on y_i

► $k(\mathbf{x}, \mathbf{x}_i)$ the kernel

do NOT depend on y_i

Definition (Kernel)

a function k from $\mathcal{X} \times \mathcal{X}$ onto $\mathbb{R}$.

semi-parametric version: given the family $q_j(\mathbf{x})$, $j = 1, p$

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i k(\mathbf{x}, \mathbf{x}_i) + \sum_{j=1}^p \beta_j q_j(\mathbf{x})$$

Kernel machines

use a kernel as a dictionary

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i k(\mathbf{x}, \mathbf{x}_i)$$

- ▶ α_i influence of example i depends on y_i
- ▶ $k(\mathbf{x}, \mathbf{x}_i)$ the kernel do NOT depend on y_i

Definition (Kernel)

a function k from $\mathcal{X} \times \mathcal{X}$ onto $\mathbb{R}$.

semi-parametric version: given the family $q_j(\mathbf{x})$, $j = 1, p$

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i k(\mathbf{x}, \mathbf{x}_i) + \sum_{j=1}^p \beta_j q_j(\mathbf{x})$$

Kernel machines

Definition (Kernel machines)

$$\mathcal{A}((\mathbf{x}_i, y_i)_{i=1,n})(\mathbf{x}) = \psi\left(\sum_{i=1}^n \alpha_i k(\mathbf{x}, \mathbf{x}_i) + \sum_{j=1}^p \beta_j q_j(\mathbf{x})\right)$$

α et β : parameters to be estimated.

Exemples

$$\mathcal{A}(x) = \sum_{i=1}^n \alpha_i (x - x_i)_+^3 + \beta_0 + \beta_1 x \quad \text{splines}$$

$$\mathcal{A}(\mathbf{x}) = \text{sign}\left(\sum_{i \in I} \alpha_i \exp^{-\frac{\|\mathbf{x} - \mathbf{x}_i\|^2}{b}} + \beta_0\right) \quad \text{SVM}$$

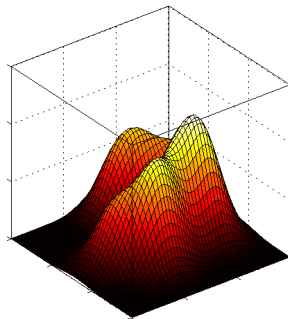
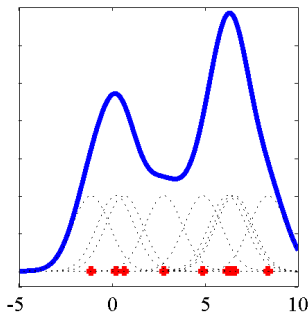
$$\mathbb{P}(y|\mathbf{x}) = \frac{1}{Z} \exp\left(\sum_{i \in I} \alpha_i \mathbb{1}_{\{y=y_i\}} (\mathbf{x}^\top \mathbf{x}_i + b)^2\right) \quad \text{exponential family}$$

example of kernel machine: the parzen estimate (1960)

assume the kernel is normalized: $\forall \mathbf{s} \in \mathcal{X}, \int_{\mathcal{X}} k(\mathbf{x}, \mathbf{s}) d\mathbf{x} = 1$

for a given data set $(\mathbf{x}_1, \dots, \mathbf{x}_i, \dots, \mathbf{x}_n)$ the parzen window estimate is

$$\hat{\mathbb{P}}(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n k(\mathbf{x}, \mathbf{x}_i)$$



- derive:
- potential based classification rule
 - the Nadaraya-Watson regression estimator

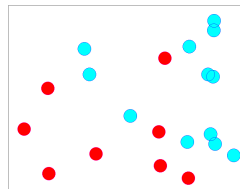
Two others early kernel machines

Bayes rule for two classes: $\text{sign}(\mathbb{P}(y \in \mathcal{C}_1|\mathbf{x}) - \mathbb{P}(y \in \mathcal{C}_2|\mathbf{x}))$

Potential based classification rule:

$$\frac{1}{n_1} \sum_{\mathbf{x}_i \in \mathcal{C}_1} k(\mathbf{x}, \mathbf{x}_i) - \frac{1}{n_2} \sum_{\mathbf{x}_i \in \mathcal{C}_2} k(\mathbf{x}, \mathbf{x}_i)$$

$$\alpha_i = \begin{cases} 1 & \text{if } \mathbf{x}_i \in \mathcal{C}_1 \\ -1 & \text{else } \mathbf{x}_i \in \mathcal{C}_2 \end{cases}$$

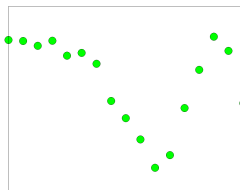


Regression function: $r(x) = \mathbb{E}(Y|X = x) = \int y \frac{\mathbb{P}(y, \mathbf{x})}{\mathbb{P}(\mathbf{x})}$

Nadaraya-Watson regression estimator:

$$\hat{f}(x) = \frac{\sum_{i=1}^n y_i k(\mathbf{x}, \mathbf{x}_i)}{\sum_{i=1}^n k(\mathbf{x}, \mathbf{x}_i)}$$

$$\alpha_i = \frac{y_i}{\sum_{i=1}^n k(\mathbf{x}, \mathbf{x}_i)}$$



what if we try to optimize the example influence α_i

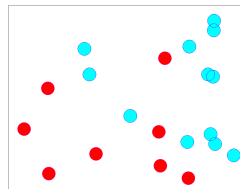
Two others early kernel machines

Bayes rule for two classes: $\text{sign}(\mathbb{P}(y \in \mathcal{C}_1|\mathbf{x}) - \mathbb{P}(y \in \mathcal{C}_2|\mathbf{x}))$

Potential based classification rule:

$$\frac{1}{n_1} \sum_{\mathbf{x}_i \in \mathcal{C}_1} k(\mathbf{x}, \mathbf{x}_i) - \frac{1}{n_2} \sum_{\mathbf{x}_i \in \mathcal{C}_2} k(\mathbf{x}, \mathbf{x}_i)$$

$$\alpha_i = \begin{cases} 1 & \text{if } \mathbf{x}_i \in \mathcal{C}_1 \\ -1 & \text{else } \mathbf{x}_i \in \mathcal{C}_2 \end{cases}$$

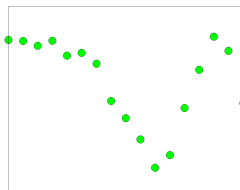


Regression function: $r(x) = \mathbb{E}(Y|X = x) = \int y \frac{\mathbb{P}(y, \mathbf{x})}{\mathbb{P}(\mathbf{x})}$

Nadaraya-Watson regression estimator:

$$\hat{f}(x) = \frac{\sum_{i=1}^n y_i k(\mathbf{x}, \mathbf{x}_i)}{\sum_{i=1}^n k(\mathbf{x}, \mathbf{x}_i)}$$

$$\alpha_i = \frac{y_i}{\sum_{i=1}^n k(\mathbf{x}, \mathbf{x}_i)}$$



what if we try to optimize the example influence α_i

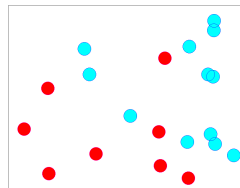
Two others early kernel machines

Bayes rule for two classes: $\text{sign}(\mathbb{P}(y \in \mathcal{C}_1|\mathbf{x}) - \mathbb{P}(y \in \mathcal{C}_2|\mathbf{x}))$

Potential based classification rule:

$$\frac{1}{n_1} \sum_{\mathbf{x}_i \in \mathcal{C}_1} k(\mathbf{x}, \mathbf{x}_i) - \frac{1}{n_2} \sum_{\mathbf{x}_i \in \mathcal{C}_2} k(\mathbf{x}, \mathbf{x}_i)$$

$$\alpha_i = \begin{cases} 1 & \text{if } \mathbf{x}_i \in \mathcal{C}_1 \\ -1 & \text{else } \mathbf{x}_i \in \mathcal{C}_2 \end{cases}$$

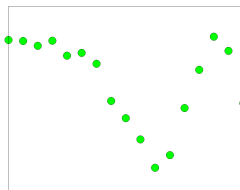


Regression function: $r(x) = \mathbb{E}(Y|X = x) = \int y \frac{\mathbb{P}(y, \mathbf{x})}{\mathbb{P}(\mathbf{x})}$

Nadaraya-Watson regression estimator:

$$\hat{f}(x) = \frac{\sum_{i=1}^n y_i k(\mathbf{x}, \mathbf{x}_i)}{\sum_{i=1}^n k(\mathbf{x}, \mathbf{x}_i)}$$

$$\alpha_i = \frac{y_i}{\sum_{i=1}^n k(\mathbf{x}, \mathbf{x}_i)}$$



what if we try to optimize the example influence α_i

Different kernel machines

▶ different kernels

kernel machines are defined for (almost) ANY kernel

kernel machines are defined for (almost) ANY input

▶ kernel machines are defined through the cost functions

▶ task dependent criterion:

- ▶ classif (SVM, K Log. reg), regression (SVR, splines)
- ▶ ranking, clustering (OCSVM), semi supervised (Trans. SVM)
- ▶ dim. reduction (KPCA, KPLS), sources separation (KICA)...

▶ penalty term

▶ sparse / non sparse : $l_0 = \{\alpha_i = 0\}$

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i k(\mathbf{x}, \mathbf{x}_i) + \sum_{j=1}^p \beta_j q_j(\mathbf{x})$$

▶ different implementations (algorithms)

introducing sparsity

linear algorithm $\rightarrow$ kernelization $\rightarrow$ sparsity



Different kernel machines

- ▶ different kernels

kernel machines are defined for (almost) ANY kernel

kernel machines are defined for (almost) ANY input

- ▶ kernel machines are defined through the cost functions

- ▶ task dependent criterion:

- ▶ classif (SVM, K Log. reg), regression (SVR, splines)

- ▶ ranking, clustering (OCSVM), semi supervised (Trans. SVM)

- ▶ dim. reduction (KPCA, KPLS), sources separation (KICA)...

- ▶ penalty term

- ▶ sparse / non sparse : $l_0 = \{\alpha_i = 0\}$

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i k(\mathbf{x}, \mathbf{x}_i) + \sum_{j=1}^p \beta_j q_j(\mathbf{x})$$

- ▶ different implementations (algorithms)

introducing sparsity

linear algorithm $\rightarrow$ kernelization $\rightarrow$ sparsity

Two paths for *kernelizing*

starting point: the linear model

$$f(\mathbf{x}) = \sum_{j=1}^d \beta_j x_j$$

the dictionary path	the <i>functional</i> path
$d = n, \phi_j(\mathbf{x}) = k(\mathbf{x}, \mathbf{x}_j)$ $f(\mathbf{x}) = \sum_{j=1}^d \beta_j \phi_j(\mathbf{x})$ $= \sum_{j=1}^n \beta_j k(\mathbf{x}, \mathbf{x}_j)$ $\min_{\beta \in \mathbf{R}^n} C(\beta)$	let $\mathcal{H}$ be a R.K.H.S. $f \in \mathcal{H}$ $\min_{f \in \mathcal{H}} C(f)$
	how to choose $\mathcal{H}$?

kernelizing algorithms

kernelizing using a dictionary

Any linear algorithm

$$\min_{\beta \in \mathbb{R}^d} J(\beta, \beta^\top \mathbf{x}_1, \dots, \beta^\top \mathbf{x}_i, \dots, \beta^\top \mathbf{x}_d)$$

can be kernelized by using $\tilde{\mathbf{x}}_i = k(\mathbf{x}, \mathbf{x}_i); i = 1, n$ as input variable (with $d = n$)

kernelizing in the dual

If the solution of a linear algorithm can be written as a linear combination of the dot product

$$\min_{\beta \in \mathbb{R}^d} J(\beta, \beta^\top \mathbf{x}_1, \dots, \beta^\top \mathbf{x}_i, \dots, \beta^\top \mathbf{x}_d)$$

can be kernelized by minimizing J with respect to a function $f \in \mathcal{H}$ where $\mathcal{H}$ is a reproducing kernel Hilbert space.



The case of linear and ridge regression

kernelizing using a dictionary

$$X = K$$

$$\min_{\beta} = \sum_{i=1}^n \left(\sum_{j=1}^n K_{ij} \beta_j - y_i \right)^2 = \|K\beta - \mathbf{y}\|^2$$

$$\beta = (K^{\top} K)^{-1} K^{\top} \mathbf{y}$$

$$\min_{\beta} = \|K\beta - \mathbf{y}\|^2 + \lambda \|\beta\|^2$$

$$\beta = (K^{\top} K + \lambda I)^{-1} K^{\top} \mathbf{y}$$

kernelizing in the dual

$$\beta^{\top} \mathbf{x}_i = f(\mathbf{x}_i)$$

$$\min_{f \in \mathcal{H}} = \sum_{i=1}^n (f(\mathbf{x}_i) - y_i)^2 = \|\mathbf{f} - \mathbf{y}\|^2$$

$$\alpha = (K)^{-1} \mathbf{y}$$

$$\min_f = \|\mathbf{f} - \mathbf{y}\|^2 + \lambda \|\mathbf{f}\|^2$$

$$\alpha = (K + \lambda I)^{-1} \mathbf{y}$$

K is often badly conditioned and $K^{\top} K$ singular !



Let's summarize

- ▶ kernel k
 - ▶ defines a relation between inputs
 - ▶ provides genericity
 - ▶ pairwise comparisons
- ▶ the definition of a kernel machine require also:
 - ▶ Hypothesis set $\mathcal{H}$
 - ▶ loss function ℓ
 - ▶ a way to get universal consistency
- ▶ efficient algorithm

